

3D RF Spatial Scanner

A full-stack network engineering project utilizing ESP32-C3 edge sensing,
and 3D telemetry visualization.

Hardware: RISC-V SoC

OS: Fedora

Stack: C++ /

Linux (KDE)

Python

Embedded Layer: Edge Sensing

Infrastructure & Tools

The sensor node utilizes an **ESP32-C3 Super Mini**. The firmware performs synchronous 802.11 Layer 2 scanning to extract BSSIDs and RSSI values.

Arduino ESP32 Core (Espressif Systems)

<https://github.com/espressif/arduino-esp32>

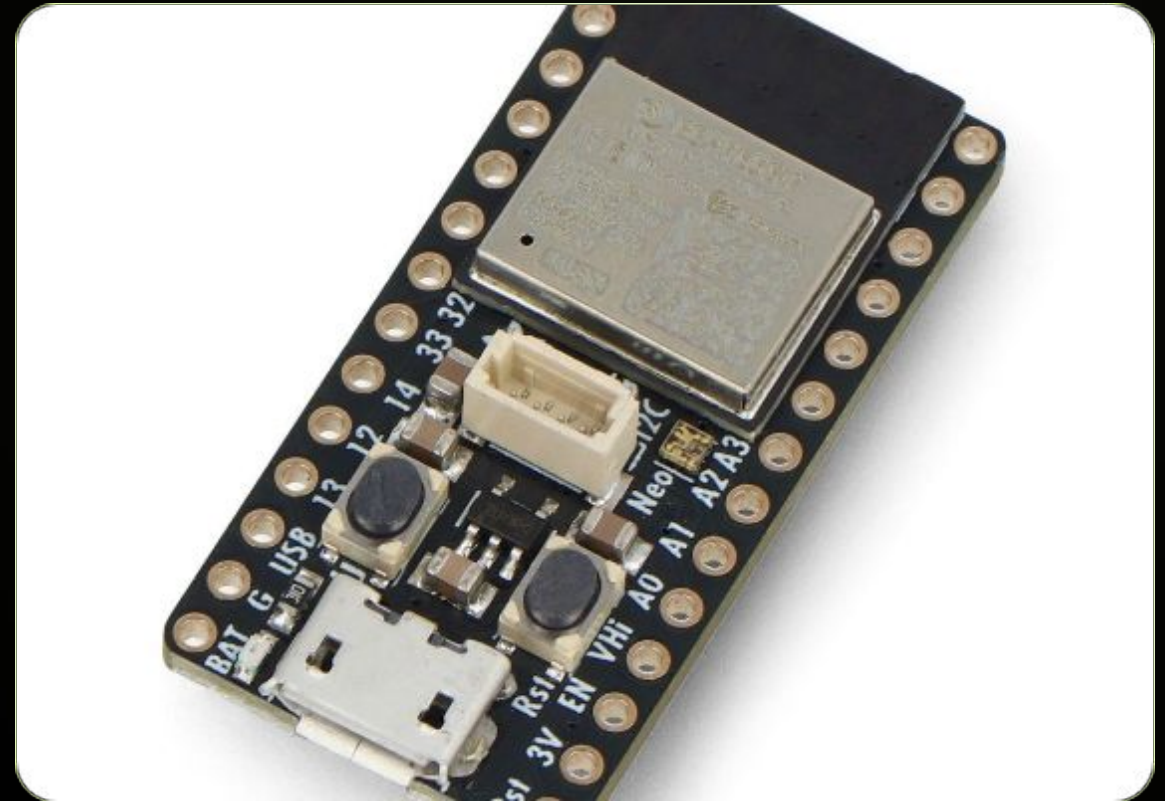
Board Manager URL:

https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

esptool (Flashing & Bootloader)

<https://github.com/espressif/esptool>

Used for RISC-V binary deployment via `esptool.py --port /dev/ttyACM0`.



Host Layer: Serial Orchestration

```
liveuser@localhost-live:~  
o dbus-launch liveinst  
st):  
up", line 82, in <module>  
p_only=True)  
*****  
te-packages/blivet/threads.py", line 49, in run_with_lock  
  
te-packages/blivet/populator/populator.py", line 444, in populate  
  
te-packages/blivet/threads.py", line 49, in run_with_lock  
  
te-packages/blivet/populator/populator.py", line 472, in _populate  
_name(new_device)  
te-packages/blivet/udev.py", line 254, in device_get_name  
RT_ENTRY_NUMBER"]  
*****  
'  
...  
ra 41 started.  
stored in /tmp during the installation  
and in second TMUX pane (ctrl+b, then press 2)  
gs from /tmp as separate text/plain attachments  
st):  
491, in <module>  
  
site-packages/pyanaconda/ui/lib/storage.py", line 367, in ignore_oemdrv_disks  
LABEL=OEMDRV", disks_only=True)  
site-packages/pyanaconda/core/storage.py", line 179, in device_matches  
.resolve_glob(full_spec)  
te-packages/blivet/udev.py", line 195, in resolve_glob  
)  
te-packages/blivet/udev.py", line 254, in device_get_name  
RT_ENTRY_NUMBER"]  
*****  
'  
...  
ra 41 started.  
stored in /tmp during the installation  
and in second TMUX pane (ctrl+b, then press 2)  
gs from /tmp as separate text/plain attachments  
st):  
491, in <module>
```

The host machine runs a high-performance Python bridge. It handles hardware permissions, serial lockouts, and spatial-telemetry

interaction

PySerial (UART Handling) <https://github.com/pyserial/pyserial>
Installed via `dnf install python3-pyserial`. Bridges `/dev/ttyACM0` to the Python data plane.

Arduino IDE (Flatpak via Flathub)
<https://flathub.org/apps/cc.arduino.arduinoide>
Permissions overridden via flatpak override `--device=all` to allow sandbox-to-hardware communication.

Core Logic: The script executes a while True loop, parsing CSV data packets from UART and appending spatial coordinates (Lat/Lon/Alt).

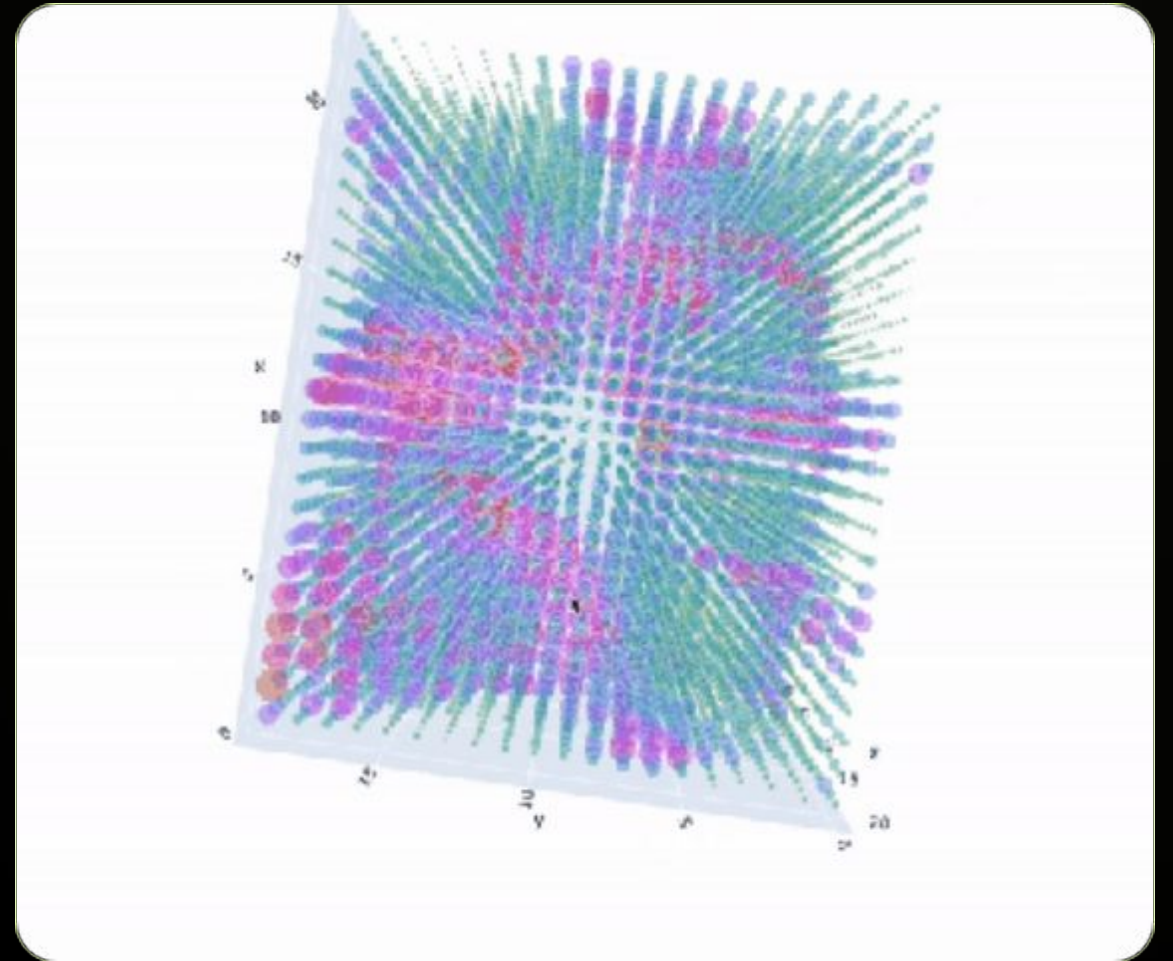
Insights: 3D Point Cloud

The Processing Pipeline

Raw telemetry logs (RSSI, BSSID, Channel) are transformed into a 3D matrix to visualize RF propagation through physical obstacles.

Tool	Functionality
Pandas	Matrix operations & CSV deserialization
Matplotlib	3D Scatter projection & color-mapped gradients
NumPy	Numerical coordination of spatial vectors

Signal Analysis: 3D clusters reveal dead zones (attenuation < -80dBm) and optimal propagation vectors for high-gain antenna placement.



Technical Portfolio Summary

Downloaded Libraries & Repository Summary

arduino-esp32 (Espressif)

pyserial (Python Serial)

pandas (Data Analysis)

matplotlib (Visualization)

esptool (Firmware Flash)

Arduino IDE (Flatpak Sandbox)

```
// Example Capture logic: int n = WiFi.scanNetworks(false, true); Serial.print(WiFi.SSID(i));  
Serial.print(","); Serial.print(WiFi.RSSI(i)); Serial.println();
```